Projet – Une application multiplateforme

React Native · En trinôme · Du 27 octobre au 17 décembre

En trinôme, vous concevez une application React Native **sur le sujet de votre choix**. Elle fonctionne avec le même code dans un navigateur de bureau et sur un téléphone. Le projet avance à chaque TP : chaque TP est un **jalon** du projet final.

Le projet en bref

- **Groupes de trois**, formés le premier jour. Un binôme est possible si l'effectif l'impose, jamais une personne seule.
- **Sujet libre**, validé par moi avant le TP2.
- Il donne la **note de projet**, qui compte pour **50 %** de la note du module. C'est la moyenne de deux notes sur 20 : la **production** du groupe et votre **soutenance individuelle**. Aucune note intermédiaire.
- **Chaque membre est propriétaire d'une partie**, dont un CRUD Firebase.
- Soutenance le 17 décembre. Rendu : dernier commit poussé **le jeudi 17 décembre avant 23h59**.

1 Ce que l'application doit contenir

Obligatoire

- **Au moins 4 écrans**, organisés avec Expo Router : des onglets, une pile et au moins une route dynamique (par exemple `/items/[id]`).
- Sur le web, **l'URL suit la navigation** : une page de détail s'ouvre directement depuis son adresse.
- Un **catalogue responsive** : une grille sur ordinateur, une liste fluide sur téléphone, avec des **filtres** dont la logique vit dans un hook.
- Une **API publique** consommée avec `fetch()`, avec les états de chargement et d'erreur.
- **Firebase** : authentification par e-mail et **trois CRUD** (créer, lire, modifier, supprimer) sur trois collections différentes, un par partie.
- Une **architecture MVC** en TypeScript : modèles typés, services, hooks contrôleurs, composants d'affichage purs.
- Une interface **lisible et cohérente** : hiérarchie visuelle, contrastes suffisants, retours visuels au toucher.
- Une **version web déployée** en ligne (GitHub Pages ou équivalent), à jour du dernier commit.
- Un `README.md` complet et **exécutable** : en suivant ses instructions sur un poste neuf, le projet se lance sans autre manipulation (voir la partie 5).

> Une application qui ne se lance pas sur le web **ou** sur Expo Go ne peut pas être évaluée. Testez les deux à chaque commit important.

Valorisé (bonus)

- Une API native du téléphone : caméra, localisation, notifications, retour haptique…
- Des animations soignées.
- Un cache ou un mode hors ligne.
- Des tests automatisés des fonctions du modèle.
- Une bibliothèque d'état (Zustand, TanStack Query…), si vous savez expliquer ce choix.

2 Un propriétaire par partie

Chaque membre est **propriétaire** d'une partie de l'application. Il la conçoit, l'écrit et en répond à la soutenance. Les deux autres l'aident et relisent son code.

Partie	Contenu	Au fil des TP
A · Catalogue	Modèle et service de l'API publique, catalogue responsive, filtres, CRUD lié au catalogue (favoris annotés, par exemple)	TP2 : modèle, service factice et catalogue. TP3 : API publique et filtres. TP4 : CRUD A
B · Navigation et compte	Structure de navigation, écran de détail, authentification, CRUD B	TP2 : modèle et service de la collection B. TP3 : navigation et détail. TP4 : authentification et CRUD B
C · Données partagées	État partagé (Context), écran groupé (`SectionList`), rafraîchissement au focus, CRUD C	TP2 : Context et deuxième écran. TP3 : `SectionList` et cycle de vie. TP4 : CRUD C

> Je note avec vous le propriétaire de chaque partie dès le premier jour. Indiquez-le aussi dans le README **dès le TP2**, avec le prénom, le nom et le compte GitHub de chacun. En binôme, répartissez les trois parties entre vous deux de la même façon. Chacun doit aussi comprendre le reste de l'application : à la soutenance, je pose une question sur la partie d'un autre membre.

3 Des idées de sujets

Ces quatre sujets s'appuient sur des API **gratuites et sans clé**. Vous pouvez en proposer un autre : il doit se prêter à un catalogue et à trois CRUD.

Sujet	API publique	Idées de CRUD Firebase (A, B, C)
Carnet de recettes	TheMealDB	Mes recettes favorites et leurs notes, mes recettes personnelles, ma liste de courses
Bibliothèque personnelle	Open Library	Mes livres à lire, mes lectures (lu, en cours) et mes avis, mes étagères thématiques
Carnet de voyage	REST Countries et Open-Meteo	Mes pays à visiter, mes voyages, les étapes d'un voyage
Garde-manger anti-gaspi	Open Food Facts	Mes produits favoris, mes produits et leurs dates de péremption, mes listes de courses

Deux groupes d'une même classe ne peuvent pas choisir le même sujet. Le premier qui me l'envoie le garde.

4 Les jalons

Quand	Jalon	Ce qui doit exister
Mar. 27 oct.	**Lancement**	Trinôme formé, sujet et parties notés avec moi, dépôt commun créé, Chris-CHEVALIER invité
Mer. 28 oct.	**TP2**	Arborescence MVC, modèles typés, service de données factices, hook contrôleur, catalogue responsive, un Context
Du 29 oct. au 24 nov.	**TP3**	Navigation complète (onglets, pile, détail), API publique, filtres, cycle de vie
Mer. 25 nov.	**TP4**	Authentification Firebase et trois CRUD
Jeu. 17 déc.	**Soutenance**	10 minutes par groupe (voir la partie 6)
Jeu. 17 déc. avant 23h59	**Rendu final**	Application complète, version web déployée à jour, README, bonus éventuels

Le 25 novembre, pendant le TP4, je fais un **point d'étape** de 5 minutes avec chaque groupe : chacun me montre où en est sa partie. Il n'est pas noté. Il me sert à repérer tôt un groupe en difficulté.

Après votre soutenance, vous pouvez encore améliorer votre projet jusqu'au soir. Je corrige la production sur le dernier commit poussé avant 23h59.

5 Le dépôt et le rendu

- Un dépôt GitHub **privé** nommé `rn-projet-nom1-nom2-nom3`, avec **Chris-CHEVALIER** invité dès le premier jour.
- **Chacun commite avec son propre compte**, régulièrement, sur la branche `main`.
- Le projet se lance avec `npm install` puis `npx expo start`, sans autre manipulation que celles du README. Aucun fichier APK ou IPA n'est demandé.

Le README contient

- le sujet, les membres du groupe (prénom, nom, compte GitHub) et le **propriétaire de chaque partie** ;
- le **lien de la version web déployée** ;
- les fonctionnalités, avec des captures d'écran sur le web et sur mobile ;
- l'API publique utilisée et le schéma des trois collections Firebase ;
- le parcours de navigation (la liste des écrans et leurs URL) ;
- les instructions de lancement, dont la configuration Firebase à fournir. Je les suis à la lettre sur un poste neuf ;
- les bonus réalisés ;
- la déclaration d'usage de l'IA.

> **Rendu final**
>
> Rien à envoyer : votre invitation me donne accès au dépôt. Je corrige le dernier commit poussé sur `main` le **jeudi 17 décembre avant 23h59**.

6 L'évaluation

Votre **note de projet** est la moyenne de deux notes sur 20 : la production du groupe et votre soutenance. Elle compte pour 50 % de la note du module, l'examen écrit pour les 50 % restants.

La production du groupe (la moitié de la note de projet)

Chaque critère vaut 1 point : il est rempli ou il ne l'est pas. Je vérifie une partie des critères automatiquement à partir de votre dépôt, le reste sur votre version web déployée et sur Expo Go.

Bloc	Critères (1 point chacun)	Points
Socle commun	Le projet se lance en suivant le README · `npx tsc --noEmit` sans erreur · `npx expo lint` sans erreur · version web déployée et à jour · MVC respecté (aucun `fetch` ni appel Firebase dans `src/app/` ou `src/components/`) · README complet · états de chargement, d'erreur et de liste vide · interface lisible et cohérente sur le web et sur mobile	8
A · Catalogue	Service de l'API (`fetch`, test de `response.ok`, réponse typée et convertie en modèle) · grille sur ordinateur et liste sur téléphone · deux filtres combinables dans un hook · CRUD A complet	4
B · Navigation et compte	Onglets, pile, route dynamique, écran `+not-found` et au moins 4 écrans · un détail s'ouvre et se recharge depuis son URL · inscription, connexion et déconnexion par e-mail · CRUD B complet	4
C · Données partagées	Un Context partagé par plusieurs écrans · une `SectionList` dont le regroupement est dans le modèle · `useFocusEffect` utilisé à bon escient et expliqué dans le README · CRUD C complet	4
Bonus (1 point par bonus réalisé, dans la limite de 20)		+2

Un CRUD est **complet** s'il permet de créer, lire, modifier et supprimer, et si chaque utilisateur ne voit que ses propres données. Un critère de fonctionnalité doit marcher **sur le web et sur mobile**.

La note de production est celle du groupe. Si la soutenance montre qu'un membre n'a pas contribué, sa note de production est réduite, jusqu'à 0.

La soutenance individuelle (l'autre moitié)

Le 17 décembre, **10 minutes par groupe**, sans IA :

- **4 minutes de démonstration** sur le web et sur mobile : chacun présente sa partie ;

- **2 minutes par membre** : vous m'expliquez un extrait de votre partie que je choisis, puis vous répondez à une question sur la partie d'un autre membre.

Critère individuel	Absent	Partiel	Complet
Démonstration de votre partie : elle fonctionne et vous la présentez clairement	0	2	4
Explication d'un extrait de votre partie : ce que fait le code, pourquoi il est écrit ainsi	0	4	8
Question sur la partie d'un autre membre : vous comprenez l'ensemble du projet	0	4	8

7 L'IA générative

- L'IA (ChatGPT, Claude, Copilot…) est **autorisée** pour le projet.
- Chaque membre doit **comprendre et savoir expliquer** chaque partie du code. Une fonctionnalité que personne ne sait expliquer pendant la soutenance n'est pas comptée.
- Déclarez votre usage dans le README : quels outils, pour quelles tâches.
- Méfiez-vous des réponses qui datent : `expo eject`, `expo-cli` global ou React Navigation configuré à la main ne correspondent pas à la version d'Expo utilisée dans ce module.